

NLP&RL: подробные ответы на экзаменационные вопросы

Конспект по курсу

1. BoW, TF-IDF

Bag-of-Words — самый простой способ представить текст численно.

Есть словарь:

$$V = \{w_1, \dots, w_{|V|}\}.$$

Каждый документ превращается в вектор длины $|V|$:

$$x_d = (x_1, \dots, x_{|V|}),$$

где

x_i = сколько раз слово w_i встретилось в документе d .

Например:

the dog is on the table

BoW:

the: 2

dog: 1

is: 1

on: 1

table: 1

Главная идея: текст представляется как мешок слов, порядок слов игнорируется.

Проблемы BoW:

- теряется порядок слов;
- огромная размерность;
- sparse-векторы;
- не кодируется семантическая близость;
- dog bites man и man bites dog имеют одинаковый BoW, хотя смысл разный.

Чтобы частично учитывать порядок, можно использовать **Bag-of-Ngrams**: не отдельные слова, а биграммы, триграммы и так далее.

Например:

The brown dog plays

биграммы:

The brown
brown dog
dog plays

Но размерность быстро растёт.

TF-IDF улучшает BoW тем, что учитывает важность слова в документе и редкость слова в корпусе.

TF:

$$tf(t, d) = f_{t,d}$$

или нормированно:

$$tf(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}.$$

IDF:

$$idf(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}.$$

Итог:

$$tfidf(t, d, D) = tf(t, d) \cdot idf(t, D).$$

Интуиция:

- если слово часто встречается в документе, оно важно для документа;
- если слово встречается во всех документах, оно неинформативно;
- редкие специфичные слова получают больший вес.

Например слова the, is, on часто встречаются почти везде, поэтому их IDF мал. А слова car, highway, neural, transformer могут быть более информативными.

2. Word embeddings. How Word2Vec is trained?

Word embedding — плотный вектор слова:

$$w \mapsto v_w \in \mathbb{R}^d,$$

где обычно:

$$d = 100, 200, 300, \dots$$

Главная идея: похожие по смыслу слова должны иметь похожие векторы.

В основе Word2Vec лежит **distributional hypothesis**:

значение слова определяется его контекстом.

То есть если cat и dog часто встречаются в похожих контекстах, их векторы должны быть близки.

Есть две основные архитектуры Word2Vec.

Skip-gram

По центральному слову предсказываем контекст.

Например предложение:

the quick brown fox jumps

Центральное слово:

brown

Окно размера 2:

the quick [brown] fox jumps

Обучающие пары:

(brown, the)

(brown, quick)

(brown, fox)

(brown, jumps)

Модель получает центральное слово и должна предсказать вероятные соседние слова.

Формально:

$$P(w_o|w_c) = \frac{\exp(u_{w_o}^\top v_{w_c})}{\sum_{w \in V} \exp(u_w^\top v_{w_c})}.$$

Здесь:

- v_{w_c} — input embedding центрального слова;
- u_{w_o} — output embedding контекстного слова;
- V — словарь.

В Word2Vec для каждого слова фактически есть два вектора:

- input vector v_w ;
- output vector u_w .

После обучения обычно используют input embeddings v_w , иногда усредняют u_w и v_w .

CBOW

Обратная задача: по контексту предсказываем центральное слово.

Например:

problems turning crises as [banking]

CBOW пытается предсказать banking.

CBOW быстрее и лучше для частых слов. Skip-gram медленнее, но лучше работает с редкими словами.

3. What is negative sampling in Word2Vec?

Полный softmax в Word2Vec дорогой:

$$P(w_o|w_c) = \frac{\exp(u_{w_o}^\top v_{w_c})}{\sum_{w \in V} \exp(u_w^\top v_{w_c})}.$$

Проблема: знаменатель требует пройти по всему словарю.

Если словарь:

$$|V| = 100000,$$

то на каждом шаге надо считать scores для 100000 слов.

Negative sampling заменяет задачу многоклассовой классификации на задачу бинарной классификации.

Для настоящей пары:

(center word, context word)

например:

(brown, fox)

модель должна сказать:

$$y = 1.$$

Потом берём несколько случайных слов, которые не являются настоящим контекстом:

(brown, table)

(brown, government)

(brown, banana)

Для них:

$$y = 0.$$

Loss для одной положительной пары:

$$\log \sigma(u_o^\top v_c) + \sum_{k=1}^K \log \sigma(-u_{n_k}^\top v_c).$$

То есть:

- положительную пару тянем ближе;
- отрицательные пары отталкиваем.

Отрицательные слова выбираются не равномерно, а по сглаженной частоте:

$$P(w_i) = \frac{f(w_i)^{3/4}}{\sum_j f(w_j)^{3/4}}.$$

Степень $3/4$ делает распределение мягче: частые слова всё ещё выбираются чаще, но не настолько часто, как при обычных частотах.

Главный смысл negative sampling:

обновляем не весь словарь,
а только 1 positive + K negatives.

Это резко ускоряет обучение.

4. How recurrent block works in RNN?

RNN обрабатывает последовательность по одному элементу.

На шаге t есть:

- текущий вход x_t ;
- предыдущее скрытое состояние h_{t-1} ;
- новое скрытое состояние h_t .

Vanilla RNN:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b).$$

Интуиция:

$$h_t$$

— память обо всём префиксе:

$$x_1, \dots, x_t.$$

Например:

I
I love
I love NLP

После каждого слова состояние обновляется.

Для классификации текста можно взять последнее состояние:

$$h_T$$

и подать в классификатор.

Для sequence labeling можно использовать каждое состояние:

$$h_t \rightarrow y_t.$$

Проблемы vanilla RNN:

- vanishing gradients;
- exploding gradients;
- плохое хранение дальнего контекста;
- последовательные вычисления, плохо параллелится.

Поэтому появились LSTM и GRU, где есть gates и более стабильный механизм памяти.

5. Attention mechanism

В классическом seq2seq без attention encoder сжимает всё исходное предложение в один вектор:

$$h_m.$$

Это bottleneck.

Attention решает проблему: decoder на каждом шаге может смотреть на все состояния encoder-a:

$$h_1, \dots, h_m.$$

Пусть decoder на шаге t имеет состояние:

$$s_t.$$

Считаем score для каждого encoder state:

$$e_{t,i} = \text{score}(s_t, h_i).$$

Например dot-product attention:

$$e_{t,i} = s_t^\top h_i.$$

Затем softmax:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}.$$

Получаем attention distribution:

$$\sum_i \alpha_{t,i} = 1.$$

Дальше строим context vector:

$$c_t = \sum_i \alpha_{t,i} h_i.$$

То есть c_t — взвешенная сумма состояний encoder-a.

Интуиция:

decoder сейчас генерирует слово

↓

attention выбирает, на какие source tokens смотреть

↓

context vector помогает предсказать следующий target token

Виды attention score:

$$e_{t,i} = s_t^\top h_i$$

dot-product;

$$e_{t,i} = s_t^\top W h_i$$

multiplicative;

$$e_{t,i} = v^\top \tanh(W_1 h_i + W_2 s_t)$$

additive attention.

Attention важен, потому что:

- снимает bottleneck;
- улучшает длинные предложения;
- даёт alignment между source и target;
- стал основой Transformer.

6. Machine translation. Encoder-decoder models

Machine translation — задача перевода:

$$x \rightarrow y,$$

где x — source sentence, y — target sentence.

В Neural Machine Translation модель напрямую оценивает:

$$P(y|x).$$

Seq2Seq состоит из encoder-a и decoder-a.

Encoder

Читает исходное предложение:

$$x = (x_1, \dots, x_m)$$

и строит состояния:

$$h_i = f(h_{i-1}, x_i).$$

В простом варианте последний вектор:

$$h_m$$

считается представлением всего source sentence.

Decoder

Генерирует перевод token за tokenом:

$$P(y|x) = \prod_{t=1}^T P(y_t | y_{<t}, x).$$

Начинает с <SOS>, заканчивает <EOS>.

Обучение идёт на параллельном корпусе:

source sentence → target sentence

Loss:

$$L = - \sum_{t=1}^T \log P(y_t | y_{<t}, x).$$

Часто используется teacher forcing: на вход decoder-a во время обучения подаётся правильный предыдущий token, а не token, который модель сама предсказала.

Проблема seq2seq без attention: bottleneck. Всё source sentence сжимается в один вектор. Поэтому для длинных предложений модель теряет детали.

Seq2Seq with attention решает это: decoder на каждом шаге обращается ко всем состояниям encoder-a.

7. How to measure quality in text generation problem

Для text generation качество можно измерять по-разному.

Perplexity / NLL / Cross-Entropy

Для language modeling часто используют negative log-likelihood:

$$L = - \sum_t \log P(x_t | x_{<t}).$$

Perplexity:

$$PPL = \exp \left(\frac{1}{T} L \right).$$

Чем ниже perplexity, тем лучше модель предсказывает текст.

Но проблема: хорошее предсказание следующего токена не всегда означает хороший generated text.

BLEU

Для перевода используют BLEU.

BLEU сравнивает generated text с reference text через n-gram precision.

$$precision_n = \frac{\text{число совпавших n-грамм}}{\text{число n-грамм в output}}.$$

Итог:

$$BLEU = BP \cdot \left(\prod_{n=1}^N precision_n \right)^{1/N}.$$

BP — brevity penalty, штраф за слишком короткий перевод.

Проблемы BLEU:

- оценивает буквальное совпадение, а не смысл;
- плох для одиночных примеров;
- не учитывает все допустимые переформулировки.

Human evaluation

Люди оценивают:

- adequacy;
- fluency;
- relevance;
- helpfulness;
- factuality;
- safety.

Это качественнее, но дорого.

Learned metrics

Например BLEURT, COMET, LLM-as-a-judge.

RL-style reward

Для sequence generation можно использовать reward:

$$R(x, \hat{y})$$

например BLEU, CIDEr, human preference score или domain-specific metric.

8. Unsupervised approach to machine translation

Здесь речь про unsupervised mapping между embedding-пространствами языков.

Идея: embedding-пространства разных языков часто имеют похожую геометрию.

Например отношения:

cat - dog - animal

и

gatto - cane - animale

могут быть похожими в разных языках.

Если есть небольшой словарь переводов, можно обучить матрицу:

$$W$$

такую, что:

$$Wx_i \approx y_i.$$

Задача:

$$W^* = \arg \min_W \|WX - Y\|_F.$$

Часто добавляют ортогональность:

$$W^\top W = I.$$

Ортогональная матрица сохраняет длины и углы, то есть не ломает геометрию embedding space.

После обучения перевод слова:

$$t = \arg \max_t \cos(Wx_s, y_t).$$

В полностью unsupervised варианте нет пар переводов.

Тогда идея такая:

- есть облако source embeddings X ;
- есть облако target embeddings Y ;
- ищем W , чтобы распределение WX стало похоже на Y ;
- можно использовать adversarial training / GAN;
- затем по ближайшим соседям уточнять словарь.

То есть модель пытается совместить два пространства, не зная заранее, что cat = gatto.

9. Self-Attention mechanism

Self-attention — attention внутри одной последовательности.

В seq2seq attention decoder смотрел на encoder.

В self-attention токены одной последовательности смотрят друг на друга.

Для каждого токена x_i строятся:

$$q_i = W^Q x_i,$$

$$k_i = W^K x_i,$$

$$v_i = W^V x_i.$$

Интуиция:

- Query — что токен ищет;
- Key — по чему токен можно найти;
- Value — какую информацию токен отдаёт.

Для токена i считаем score со всеми токенами j :

$$\text{score}(i, j) = q_i^\top k_j.$$

Масштабируем:

$$\frac{q_i^\top k_j}{\sqrt{d_k}}.$$

Softmax:

$$\alpha_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{d_k})}{\sum_l \exp(q_i^\top k_l / \sqrt{d_k})}.$$

Выход:

$$z_i = \sum_j \alpha_{ij} v_j.$$

Матричная форма:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V.$$

$$Attention(Q, K, V) = softmax \left(\frac{QK^\top}{\sqrt{d_k}} \right) V.$$

Self-attention полезен, потому что:

- каждый токен может напрямую учитывать любой другой токен;
- path length между позициями $O(1)$;
- хорошо параллелится;
- строит контекстуальные представления.

10. Transformer architecture

Классический Transformer состоит из:

Encoder stack

Decoder stack

Encoder block

Содержит:

Multi-Head Self-Attention

Feed-Forward Network

Residual connections

LayerNorm

Схема:

$$X \rightarrow SelfAttention \rightarrow Z \rightarrow FFN \rightarrow R.$$

Self-attention смешивает информацию между токенами.

FFN обрабатывает каждую позицию независимо:

$$FFN(x) = W_2 \sigma(W_1 x + b_1) + b_2.$$

Decoder block

Содержит:

Masked Self-Attention
Encoder-Decoder Attention
Feed-Forward Network
Residual + LayerNorm

Masked self-attention нужен, чтобы decoder не смотрел в будущие target tokens.

Маска:

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

Encoder-decoder attention:

- queries из decoder-a;
- keys и values из encoder-a.

То есть decoder спрашивает:

на какую часть source sentence смотреть сейчас?

Positional encoding

Self-attention сам по себе не знает порядок слов.

Поэтому:

$$\tilde{x}_i = x_i + p_i.$$

Классический sinusoidal positional encoding:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right),$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right).$$

Multi-head attention

Несколько attention-голов:

$$head_r = Attention(Q_r, K_r, V_r).$$

$$MultiHead = Concat(head_1, \dots, head_H)W^O.$$

Разные головы могут ловить разные зависимости.

Главный плюс Transformer:

- параллелизация;
- дальние зависимости;
- контекстуальные embeddings;

- короткий путь между токенами.

Главный минус:

$$O(n^2)$$

по длине последовательности из-за матрицы attention.

11. BERT pretraining. MLM and NSP objectives

BERT = **Bidirectional Encoder Representations from Transformers**.

Это encoder-only Transformer.

Он хорошо подходит для задач понимания текста, потому что encoder видит весь текст сразу: и левый, и правый контекст.

MLM — Masked Language Modeling

Берём текст:

The animal did not cross the street because it was tired.

Маскируем часть токенов:

The animal did not cross the [MASK] because it was tired.

Модель должна восстановить:

street

Objective:

$$-\log P(x_i | x_{\setminus i}).$$

Смысл: модель научится понимать слово по контексту с обеих сторон.

Это отличается от GPT, где модель предсказывает следующий токен только по левому контексту:

$$P(x_t | x_{<t}).$$

NSP — Next Sentence Prediction

BERT получает два предложения A и B .

Нужно определить:

$$y \in \{IsNext, NotNext\}.$$

Пример:

A: I bought a dog.

B: It is very friendly.

IsNext.

Другой пример:

A: I bought a dog.

B: The Eiffel Tower is in Paris.

NotNext.

NSP должен был помочь задачам, где важны отношения между предложениями:

- question answering;
- natural language inference;
- entailment.

12. BERT finetuning. Transfer learning paradigm. Finetuning strategies

Transfer learning:

pretraining на огромном неразмеченном корпусе

↓

fine-tuning на конкретной задаче

Pretraining учит общие языковые представления. Fine-tuning адаптирует модель под downstream task.

Text classification

В начало добавляется токен:

[CLS]

После BERT берём его hidden state:

$$h_{CLS}.$$

Классификатор:

$$\hat{y} = \text{softmax}(Wh_{CLS} + b).$$

Используется для sentiment analysis, spam detection, topic classification.

Token classification

Для NER или POS-tagging используем выход каждого токена:

$$h_1, \dots, h_n.$$

Для каждого токена:

$$y_i = \text{softmax}(Wh_i + b).$$

Question Answering

Для extractive QA модель предсказывает начало и конец ответа:

$$p_{start}, p_{end}.$$

Ответ — span в тексте.

Feature extraction

Можно не дообучать BERT.

Тогда:

text → BERT → hidden states → другая модель

Плюс: быстро и дёшево.

Минус: обычно хуже fine-tuning.

Fine-tuning всей модели

Обновляются все веса BERT и task-specific head.

Плюс: лучше качество.

Минус: дороже, риск overfitting на маленьких данных.

Partial freezing

Можно заморозить нижние слои и обучать только верхние + head.

Полезно, если мало данных или ресурсов.

13. Contrastive learning paradigm. Examples

Contrastive learning учит representation space так, чтобы похожие объекты были близко, а непохожие далеко.

Есть:

- anchor x ;
- positive sample x^+ ;
- negative samples $x_1^-, \dots, x_{N-1}^-$.

Хотим:

$$\text{score}(f(x), f(x^+))$$

большой, а

$$\text{score}(f(x), f(x^-))$$

маленький.

InfoNCE:

$$L = -\log \frac{\exp(s(x, x^+))}{\exp(s(x, x^+)) + \sum_i \exp(s(x, x_i^-))}.$$

Часто score — cosine similarity:

$$\text{sim}(u, v) = \frac{u^\top v}{\|u\| \|v\|}.$$

C temperature:

$$L = -\log \frac{\exp(\text{sim}(z, z^+)/\tau)}{\exp(\text{sim}(z, z^+)/\tau) + \sum_i \exp(\text{sim}(z, z_i^-)/\tau)}.$$

Примеры.

SimCLR

Positive pair — две augmentations одной картинки.

image
↓
crop/color jitter/blur
↓
view 1

image
↓
another augmentation
↓
view 2

Encoder:

$$h = f(x).$$

Projection head:

$$z = g(h).$$

Loss применяется к z , а downstream использует h .

MoCo

Использует очередь negatives и два encoder-а:

- query encoder;
- key encoder.

Key encoder обновляется momentum update:

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q.$$

Это позволяет иметь много negatives без огромного batch size.

CPC

Contrastive Predictive Coding работает с последовательностями.

Кодируем:

$$z_t = g_{enc}(x_t).$$

Агрегируем контекст:

$$c_t = g_{ar}(z_{\leq t}).$$

Предсказываем будущий код:

$$z_{t+k}$$

contrastive способом: отличить правильное будущее от неправильных samples.

14. How CLIP was trained to produce joined embeddings for text and images?

CLIP обучается на парах:

(image, caption)

Есть два encoder-a:

$$v_i = f_{image}(image_i),$$

$$t_i = f_{text}(caption_i).$$

Оба вектора попадают в общее embedding-пространство.

В batch есть N изображений и N подписей.

Считаем матрицу сходства:

$$S_{ij} = sim(v_i, t_j).$$

Правильные пары лежат на диагонали:

$$(i, i).$$

Все остальные пары — negatives.

Loss симметричный:

- для каждой картинке выбрать правильный текст;
- для каждого текста выбрать правильную картинку.

То есть CLIP учит:

image of cat $\approx$ "a photo of a cat"

image of airplane far from "a photo of a cat"

После обучения появляется общее image-text пространство.

Zero-shot classification

Для картинки создаём текстовые prompts:

a photo of a cat

a photo of a dog

a photo of a car

Кодируем их text encoder-ом.

Кодируем картинку image encoder-ом.

Выбираем текст с максимальным cosine similarity.

Это позволяет классифицировать изображения без обучения отдельного классификатора.

15. Reinforcement Learning problem statement, how it differs from Supervised Learning

В supervised learning есть датасет:

$$(x_i, y_i).$$

Модель учится приближать правильные ответы:

$$f(x_i) \approx y_i.$$

Данные фиксированы. Модель не влияет на то, какие данные она увидит.

В reinforcement learning есть агент и среда.

На шаге t :

- агент находится в состоянии s_t ;
- выбирает действие a_t ;
- получает reward r_t ;
- среда переходит в s_{t+1} .

Цель:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi}[R].$$

Главное отличие:

в RL действия агента влияют на будущие состояния и будущие данные.

В supervised learning модель просто учится на reference answers.

В RL агент учится через trial and error.

Также в RL есть exploration-exploitation tradeoff:

- exploitation — использовать уже известные хорошие действия;
- exploration — пробовать новые действия.

16. Markov property

Markov property означает:

будущее зависит от прошлого только через текущее состояние.

Формально:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1}|s_t, a_t).$$

То есть если состояние s_t правильно описывает всю нужную информацию, то история до s_t уже не нужна.

MDP задаётся:

$$(S, A, P, R),$$

где:

- S — states;
- A — actions;
- $P(s'|s, a)$ — transition probability;
- $R(s, a)$ — reward.

Пример: шахматы. Если состояние доски полностью известно, то не важно, какими ходами мы пришли к этой позиции. Возможные следующие состояния зависят только от текущей доски и выбранного хода.

17. Reward discounting. Cumulative reward

Без discounting total reward:

$$R = \sum_{t=0}^T r_t.$$

Для бесконечных процессов сумма может расходиться.

Поэтому вводят discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

Или для конечного эпизода:

$$G_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}.$$

Где:

$$\gamma \in [0, 1].$$

Если $\gamma = 0$, агент думает только о немедленной награде.

Если $\gamma \approx 1$, агент учитывает далёкое будущее.

Рекурсивная форма:

$$G_t = r_t + \gamma G_{t+1}.$$

Интуиция:

reward сейчас важнее,
reward далеко в будущем менее важен.

Discounting также помогает математически стабилизировать бесконечные задачи.

18. Value function. How to compute for session?

Value function показывает, насколько хорошим является состояние при следовании policy π .

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | s_t = s].$$

То есть:

expected discounted return, если стартуем из состояния s и дальше следуем policy π .

Для одной конкретной session / trajectory:

$$s_0, a_0, r_0, s_1, a_1, r_1, \dots$$

можно посчитать returns назад:

$$G_T = r_T,$$

$$G_t = r_t + \gamma G_{t+1}.$$

Например rewards:

$$r_0 = 1$$

$$r_1 = 2$$

$$r_2 = 3$$

$$\gamma = 0.9.$$

Тогда:

$$G_2 = 3,$$

$$G_1 = 2 + 0.9 \cdot 3 = 4.7,$$

$$G_0 = 1 + 0.9 \cdot 4.7 = 5.23.$$

Если в разных sessions одно и то же состояние s встречалось много раз, можно усреднить returns:

$$V^\pi(s) \approx \frac{1}{N} \sum_i G_i(s).$$

Это Monte Carlo estimate.

19. Q-function. How to learn to estimate? Two ways

Q-function:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a].$$

Она говорит:

какой expected return, если в состоянии s сначала сделать действие a , а потом следовать policy π .

Оптимальная Q-function:

$$Q^*(s, a)$$

показывает, насколько хорошо действие a при оптимальном дальнейшем поведении.

Policy из Q:

$$\pi(s) = \arg \max_a Q(s, a).$$

Способ 1: Monte Carlo

Собираем trajectories.

Для каждого появления пары (s, a) считаем полный return:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

Потом усредняем:

$$Q(s, a) \approx \frac{1}{N} \sum_i G_i.$$

Плюсы:

- просто;
- не требует модели среды.

Минусы:

- нужно ждать конца эпизода;
- высокая дисперсия.

Способ 2: Temporal Difference / Q-learning

Используем Bellman update.

Target:

$$\hat{Q}(s, a) = r + \gamma \max_{a'} Q(s', a').$$

Update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)].$$

Скобка:

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

называется TD error.

Плюсы:

- можно обновляться после каждого transition;
- не нужно ждать конца эпизода.

Минусы:

- bootstrapping может давать bias;
- обучение может быть нестабильным с function approximation.

20. How policy gradient estimates the reward gradient w.r.t. policy parameters?

Policy gradient напрямую оптимизирует policy:

$$\pi_{\theta}(a|s).$$

Objective:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[G(\tau)].$$

Проблема: reward может быть недифференцируемым, а действия могут быть дискретными.

Используется log-derivative trick:

$$\nabla_{\theta} \pi_{\theta}(a|s) = \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s).$$

Для one-step:

$$J = \sum_a \pi_{\theta}(a|s) R(s, a).$$

Градиент:

$$\nabla_{\theta} J = \sum_a \nabla_{\theta} \pi_{\theta}(a|s) R(s, a).$$

Подставляем trick:

$$\nabla_{\theta} J = \sum_a \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) R(s, a).$$

То есть:

$$\nabla_{\theta} J = \mathbb{E}_{a \sim \pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a|s) R(s, a)].$$

Для trajectory:

$$\nabla_{\theta} J \approx \frac{1}{N} \sum_{i=1}^N \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) G_t^i.$$

Интуиция:

- если действие привело к высокому return, увеличиваем вероятность этого действия;
- если к низкому return, уменьшаем.

REINFORCE:

sample trajectories under current policy
 compute returns
 multiply log-prob gradients by returns
 gradient ascent

Это on-policy метод.

21. What is baseline in policy gradient?

Baseline — величина $b(s)$, которую можно вычитать из return/Q-value:

$$\nabla_{\theta} J = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a|s)(Q(s, a) - b(s))].$$

Главное условие:

$$b(s)$$

не должен зависеть от action a .

Почему expected gradient не меняется?

$$\mathbb{E}_{a \sim \pi}[\nabla_{\theta} \log \pi_{\theta}(a|s)b(s)] = b(s) \sum_a \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s).$$

Но:

$$\pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) = \nabla_{\theta} \pi_{\theta}(a|s).$$

Значит:

$$b(s) \sum_a \nabla_{\theta} \pi_{\theta}(a|s) = b(s) \nabla_{\theta} \sum_a \pi_{\theta}(a|s) = b(s) \nabla_{\theta} 1 = 0.$$

То есть baseline не меняет математическое ожидание градиента.

Зачем он нужен?

Чтобы уменьшить variance.

Хороший baseline:

$$b(s) = V(s).$$

Тогда:

$$Q(s, a) - V(s) = A(s, a).$$

Это advantage.

Интерпретация:

- $A(s, a) > 0$: действие лучше среднего для этого состояния;
- $A(s, a) < 0$: действие хуже среднего;
- $A(s, a) = 0$: действие примерно обычное.

В SCST baseline — reward greedy output:

$$A(s, a) = R(s, a) - R(s, a_{\text{inference}}(s)).$$

22. Why direct training on Likelihood / CrossEntropy minimization does not work well enough in language modeling?

В supervised seq2seq training модель максимизирует likelihood reference output:

$$\max_{\theta} \log P_{\theta}(y|x) = \max_{\theta} \sum_t \log P_{\theta}(y_t|x, y_{<t}).$$

Это удобно и стабильно, но есть несколько проблем.

1. Training-inference mismatch / exposure bias

Во время обучения используется teacher forcing:

input decoder-а: правильные previous tokens

То есть:

$$P(y_t|x, y_{<t}^{\text{true}}).$$

А во время inference модель получает свои предыдущие предсказания:

$$P(y_t|x, \hat{y}_{<t}).$$

Если модель ошиблась на раннем шаге, дальше она оказывается в состоянии, которого почти не видела при обучении.

Ошибка накапливается.

Это называется distribution shift:

training: prefixes from reference

inference: prefixes from model

2. Много правильных ответов

Для перевода, диалогов, summarization часто существует много корректных outputs.

Например:

I am fine.

I'm okay.

Doing well.

Все могут быть правильными.

Cross-entropy обучает модель повторять один reference. Если модель выдаёт другой хороший ответ, loss может быть большим.

3. Token-level objective не равен sequence-level quality

Cross-entropy оптимизирует каждый следующий токен.

Но качество текста часто определяется всей последовательностью:

- BLEU;
- ROUGE;
- CIDEr;
- human preference;
- factual correctness;
- helpfulness.

Модель может иметь хороший token-level likelihood, но плохой sequence-level score.

4. Cross-entropy не оптимизирует напрямую метрику

Например в image captioning нас интересует CIDEr, а в translation — BLEU.

Но MLE оптимизирует:

$$\log P(y|x),$$

а не:

$$BLEU(\hat{y}, y).$$

RL fine-tuning позволяет напрямую оптимизировать:

$$\mathbb{E}[R(x, \hat{y})].$$

5. Проблема dull / generic responses

В диалогах likelihood training часто поощряет безопасные усреднённые ответы:

I don't know.

Okay.

That's interesting.

Потому что они часто встречаются и подходят ко многим контекстам.

Практический вывод

Поэтому часто используют схему:

cross-entropy supervised pretraining

↓

RL fine-tuning / preference optimization / SCST

Cross-entropy нужен для cold start: случайная policy генерирует мусор, и RL с нуля плохо работает.

RL нужен, чтобы оптимизировать настоящую sequence-level reward и уменьшить mismatch между training и inference.